

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN

NGÔ TIẾN SỸ – 23521367
NGUYỄN VĂN NAM – 23520982

BÁO CÁO ĐỒ ÁN
XÂY DỰNG KIẾN TRÚC DATA LAKEHOUSE THỜI GIAN
THỰC CHO HỆ THỐNG GIAO THÔNG THÔNG MINH
TRÊN CỤM KUBERNETES

**Building a Real-Time Data Lakehouse Architecture for
Intelligent Transportation Systems on a Kubernetes Cluster**

CHUYÊN NGÀNH HỆ THỐNG THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN
THS. NGUYỄN HỒ DUY TRÍ

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN

NGÔ TIẾN SỸ – 23521367
NGUYỄN VĂN NAM – 23520982

BÁO CÁO ĐỒ ÁN
XÂY DỰNG KIẾN TRÚC DATA LAKEHOUSE THỜI GIAN
THỰC CHO HỆ THỐNG GIAO THÔNG THÔNG MINH
TRÊN CỤM KUBERNETES

**Building a Real-Time Data Lakehouse Architecture for
Intelligent Transportation Systems on a Kubernetes Cluster**

CHUYÊN NGÀNH HỆ THỐNG THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN
THS. NGUYỄN HỒ DUY TRÍ

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Nhóm thực hiện xin trân trọng cảm ơn ThS. Nguyễn Hồ Duy Trí đã hướng dẫn, góp ý và định hướng trong quá trình xây dựng đề tài. Những trao đổi về kiến trúc dữ liệu, xử lý luồng, triển khai trên Kubernetes và cách diễn giải kết quả thực nghiệm giúp nhóm hoàn thiện báo cáo theo hướng rõ ràng và có khả năng kiểm chứng hơn.

Nhóm cũng xin cảm ơn Trường Đại học Công nghệ Thông tin, Đại học Quốc gia TP. Hồ Chí Minh và Khoa Hệ thống Thông tin đã tạo môi trường học tập để nhóm có thể tiếp cận các kiến thức về cơ sở dữ liệu phân tán, dữ liệu lớn, hạ tầng Kubernetes, GitOps và quan sát hệ thống. Các kiến thức này là nền tảng để nhóm thiết kế, triển khai và đánh giá artifact Continuum trong phạm vi đề án.

Do giới hạn thời gian và hạ tầng thực nghiệm, báo cáo khó tránh khỏi thiếu sót. Nhóm mong nhận được góp ý từ giảng viên và các bạn để tiếp tục cải thiện hệ thống trong các phiên bản sau.

TP. Hồ Chí Minh, 2026

Nhóm thực hiện

MỤC LỤC

Tóm tắt báo cáo.....	1
Chương 1. Mở đầu	2
1.1 Đặt vấn đề	2
1.2 Lý do chọn đề tài	3
1.3 Mục tiêu nghiên cứu	3
1.4 Đối tượng và phạm vi	3
1.5 Câu hỏi nghiên cứu.....	4
1.6 Đóng góp và cấu trúc báo cáo	4
Chương 2. Tổng quan và cơ sở lý thuyết.....	6
2.1 Data Lakehouse và Apache Iceberg	6
2.2 Stream processing, state và materialized view	6
2.3 Broker Kafka-compatible và RisingWave.....	7
2.4 Blue/Green MV swap	7
2.5 GitOps và observability	8
2.6 Ursa và nghiên cứu liên quan	9
Chương 3. Nghiên cứu thực nghiệm và kiến trúc hệ thống	10
3.1 Phương pháp nghiên cứu.....	10
3.2 Dữ liệu và chuẩn bị luồng.....	10
3.3 Kiến trúc tổng thể	11
3.4 Topology triển khai	12
3.5 Đối tượng SQL và Blue/Green design	13
3.6 Thang đo đánh giá.....	14
3.7 Nguồn bằng chứng và khả năng truy vết	14
Chương 4. Kết quả thực nghiệm, đánh giá và bàn luận	15
4.1 Môi trường và phiên bản	15
4.2 Cluster, GitOps và mức sẵn sàng pipeline	15
4.3 Phân tích chất lượng dữ liệu nguồn.....	16
4.4 Replay ingest và đầu ra lakehouse	17
4.5 Blue/Green MV cutover	18

4.6	Tính toán vận quan sát được và tài nguyên.....	19
4.7	Tổng hợp đánh giá.....	21
4.8	Thảo luận kết quả	21
4.9	Hạn chế	22
Chương 5.	Kết luận	23
Chương 6.	Hướng phát triển.....	24
Tài liệu tham khảo		25
Phụ lục A.	SQL cốt lõi.....	27
A.1	Source, lookup và baseline MV	27
A.2	Green MV và swap.....	27
A.3	Iceberg sink.....	28
Phụ lục B.	Cấu hình vận hành trích yếu	28

DANH MỤC HÌNH

Hình 3.1:	Kiến trúc và luồng quy trình Continux.....	11
Hình 4.1:	Dashboard hiệu năng streaming sau lượt replay/cutover. Consumer lag hiển thị 0 trong cửa sổ ảnh.	18
Hình 4.2:	Dashboard cutover: green ready, query errors bằng 0 và restart bằng 0; panel duration làm tròn giá trị nhỏ hơn một giây.	19
Hình 4.3:	Dashboard tính toán vẹn dữ liệu sau swap; mismatch được diễn giải theo thay đổi logic.....	20
Hình 4.4:	Dashboard resource utilization trong cửa sổ quan sát thực nghiệm.	20

DANH MỤC BẢNG

Bảng 1.1:	Câu hỏi nghiên cứu và bằng chứng trả lời	4
Bảng 2.1:	Vai trò các lớp công nghệ trong Continux	9
Bảng 3.1:	Nguồn dữ liệu thực nghiệm.....	11
Bảng 3.2:	Topology cụm K3s thực nghiệm	12
Bảng 3.3:	Đối tượng dữ liệu cốt lõi.....	13
Bảng 3.4:	Nhóm thang đo và cách diễn giải	14
Bảng 4.1:	Phiên bản phần mềm được ghi trong repo	15
Bảng 4.2:	Kết quả hạ tầng và pipeline.....	15
Bảng 4.3:	Phân tích chất lượng bộ dữ liệu NYC TLC Yellow Taxi tháng 2026-03 sau chuyển đổi JSONL	16
Bảng 4.4:	Kết quả Blue/Green MV swap	18
Bảng 4.5:	Tổng hợp kết luận theo bằng chứng	21
Bảng 4.6:	Giới hạn bằng chứng và tác động tới kết luận	22

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
API	Application Programming Interface – giao diện lập trình ứng dụng
CSV	Comma-Separated Values – định dạng dữ liệu phân tách bằng dấu phẩy
DSR	Design Science Research – phương pháp nghiên cứu khoa học thiết kế
GitOps	Mô hình vận hành dùng Git làm nguồn trạng thái mong muốn
HA	High Availability – tính sẵn sàng cao
ITS	Intelligent Transportation System – hệ thống giao thông thông minh
JSONL	JSON Lines – định dạng mỗi dòng là một JSON độc lập
K3s	Bản phân phối Kubernetes gọn nhẹ của SUSE
MV	Materialized View – khung nhìn vật lý
PVC	PersistentVolumeClaim – yêu cầu cấp phát lưu trữ bền vững trong Kubernetes
S3	Simple Storage Service – giao diện lưu trữ đối tượng tương thích Amazon S3
SDG	Sustainable Development Goal – Mục tiêu Phát triển Bền vững
SQL	Structured Query Language – ngôn ngữ truy vấn có cấu trúc
TLC	Taxi and Limousine Commission – Ủy ban Taxi và Limousine Thành phố New York

TÓM TẮT BÁO CÁO

Nghiên cứu này xây dựng và đánh giá **Continuix**, một kiến trúc Data Lakehouse thời gian thực cho dữ liệu giao thông thông minh trên cụm K3s ba server. Hệ thống chuyển đổi dữ liệu NYC TLC Yellow Taxi từ kiểu Parquet sang kiểu JSONL, sau đó phát lại các bản ghi thành luồng sự kiện bằng Vector, đẩy các sự kiện vào Redpanda như một lớp trung gian tiếp nhận và phân phối luồng, xử lý bằng truy vấn SQL theo luồng (streaming SQL) trong RisingWave, và ghi kết quả vào các bảng Apache Iceberg được lưu trên MinIO; đồng thời hệ thống được quản lý triển khai bằng Argo CD và quan sát bằng VictoriaMetrics/Grafana. Artifact (hiện vật và bộ thành phần thực nghiệm của hệ thống) được kiểm chứng qua một lượt phát lại có kiểm soát và kịch bản chuyển đổi khung nhìn vật lý (materialized view cutover) giữa nhánh cũ và nhánh mới: public MV (khung nhìn vật lý công khai phục vụ truy vấn bên ngoài) được chuyển sang logic green; vòng lặp truy vấn không ghi nhận lỗi, RisingWave không tự khởi động lại, và hệ thống ghi nhận thời gian chuyển đổi là $0,145226$ s. Kết quả cho thấy khả năng cập nhật logic truy vấn công khai mà không ghi nhận gián đoạn trong cửa sổ thử nghiệm, đồng thời pipeline tạo được đầu ra Iceberg quan sát được trên MinIO.

Từ khóa: Data Lakehouse, stream processing, Materialized View, Blue/Green cutover, RisingWave, Apache Iceberg, GitOps, K3s, ITS.

Chương 1. Mở đầu

1.1. Đặt vấn đề

Dữ liệu giao thông đô thị phát sinh liên tục và mang ý nghĩa vận hành tức thời: mỗi chuyến xe bổ sung thông tin về điểm đón, quãng đường, chi phí và nhu cầu theo khu vực. Với hệ thống giao thông thông minh, gọi tắt là ITS (*Intelligent Transportation System*), việc lưu dữ liệu chỉ để phân tích theo batch là chưa đủ; người vận hành cần theo dõi tổng hợp cập nhật gần thời gian thực và cần thay đổi quy tắc phân tích khi yêu cầu chất lượng dữ liệu hoặc chính sách nghiệp vụ thay đổi. Yêu cầu “cập nhật gần thời gian thực” ở đây nghĩa là kết quả phải phản ánh sự kiện gần nhất trong khoảng từ giây đến vài chục giây, thay vì sau một lần chạy batch định kỳ.

Trong xử lý batch, hệ thống gom một lượng dữ liệu rồi tính toán theo lịch định trước; kết quả vì vậy thường phản ánh quá khứ sau một độ trễ. Ngược lại, xử lý luồng (*stream processing*) xem mỗi bản ghi mới là một *event*, tức một thông điệp có thể cập nhật kết quả đang phục vụ ngay khi đến. Một thành phần then chốt cho phép thực hiện điều này là *materialized view* (MV), tức một bảng kết quả truy vấn được hệ thống tự cập nhật khi dữ liệu nguồn thay đổi; người đọc truy vấn MV thay vì tính lại từ dữ liệu thô. Sự khác biệt giữa batch và streaming đặc biệt quan trọng với ITS, vì một thống kê khu vực đang tăng tải chỉ hữu ích cho vận hành nếu có thể được quan sát đủ sớm và nếu logic thống kê có thể sửa đổi an toàn khi đang phục vụ.

Data Lakehouse là mô hình kiến trúc kết hợp tính linh hoạt của *data lake* (lưu trữ tệp mở trên object storage chi phí thấp) với khả năng quản trị bảng và phân tích có cấu trúc kiểu *data warehouse*, được đề xuất chính thức trong nghiên cứu trước đây [3]. Trong khi đó, xử lý luồng đưa thêm vào chuỗi vận hành các thành phần như *broker* (lớp trung gian lưu và phân phối thông điệp), engine tính toán, *state store* (nơi giữ trạng thái trung gian), *sink* (đầu ra ghi xuống một hệ thống đích) và lớp quan sát. Bài báo Ursa [8] chỉ ra chi phí và độ phức tạp khi streaming và lakehouse bị tách rời, đồng thời đề xuất một engine lakehouse-native cho Kafka. Tại lớp ứng dụng phân tích vẫn còn một câu hỏi thực hành chưa được trả lời rộng rãi: khi logic tổng hợp đang phục vụ truy vấn phải thay đổi, việc chuyển đổi có thể thực hiện mà không làm xuất hiện lỗi truy vấn hay không?

1.2. Lý do chọn đề tài

Continux khảo sát câu hỏi đặt ra ở mục trước trong bối cảnh khả thi với phòng thí nghiệm: một cụm Kubernetes nhẹ tên K3s, dữ liệu taxi mở của Sở Taxi và Dịch vụ vận tải New York (NYC TLC) và các công nghệ nguồn mở thông dụng. K3s là một bản phân phối Kubernetes gọn nhẹ do SUSE phát hành, có khả năng chạy trên máy có tài nguyên hạn chế; lựa chọn K3s giúp đề tài tái hiện được nhiều khái niệm cốt lõi của Kubernetes mà không yêu cầu hạ tầng đám mây quy mô lớn. Đề tài có liên hệ với các Mục tiêu Phát triển Bền vững (*Sustainable Development Goal*, SDG) của Liên Hợp Quốc: SDG 8 qua nhu cầu hạ tầng dữ liệu hỗ trợ dịch vụ di chuyển hiệu quả; SDG 9 qua thử nghiệm hạ tầng số và đổi mới công nghệ; và SDG 11 qua khả năng quan sát dữ liệu di chuyển phục vụ đô thị bền vững [14, 15, 16]. Liên hệ này được sử dụng như động lực ứng dụng, không phải một phép đánh giá định lượng tác động SDG.

Ursa nhấn mạnh đường ghi lakehouse-native và bảo đảm xử lý của streaming engine. Continux không tái hiện benchmark chi phí của Ursa; thay vào đó, đề tài khảo sát lớp vận hành (operational layer) phía trên ingest, bao gồm GitOps cho triển khai, dashboard cho quan sát, và một kỹ thuật thay đổi logic truy vấn dùng hai materialized view song song cùng phép *swap* tên trong RisingWave. RisingWave là streaming database SQL được trình bày chi tiết ở Chương 2.

1.3. Mục tiêu nghiên cứu

Nghiên cứu hướng tới bốn mục tiêu phối hợp với nhau, đi từ thiết kế đến đánh giá:

- Thiết kế pipeline streaming lakehouse từ dữ liệu NYC TLC tới đầu ra Apache Iceberg trên object storage.
- Triển khai pipeline trên cụm K3s ba server có HA ở lớp control plane và etcd, tách rõ data plane, control/observability plane và node quorum-only.
- Thực hiện Blue/Green cutover tại lớp public materialized view, đồng thời quan sát thời gian chuyển đổi, lỗi truy vấn và trạng thái workload.
- Đánh giá artifact bằng đúng bằng chứng đã thu thập, không mở rộng kết luận vượt quá dữ liệu đo được.

1.4. Đối tượng và phạm vi

Đối tượng xử lý là dữ liệu công khai NYC TLC Yellow Taxi tháng 2026-03 và bảng tra cứu khu vực Taxi Zone Lookup. Tập Parquet nguồn, một định dạng cột nén tối ưu

Bảng 1.1: Câu hỏi nghiên cứu và bằng chứng trả lời

ID	Câu hỏi	Bằng chứng	Kết luận trong phạm vi đo
RQ1	Chuyển public logic mà không ghi nhận lỗi truy vấn trong cửa sổ swap?	Runbook duration 0,145226 s; query errors 0.	Có, ở lớp truy vấn và trong cửa sổ quan sát.
RQ2	Pipeline có tạo đầu ra lakehouse quan sát được?	MinIO có data, equality-delete, position-delete Parquet và metadata.	Có, sau lượt replay.
RQ3	Cụm nhỏ có vận hành được kịch bản thiết kế?	3/3 node, 5/5 PVC, 23/23 workload.	Có, cho lượt đo báo cáo.

cho phân tích, được chuyển thành JSONL gồm 3.952.451 dòng; mỗi dòng JSONL là một bản ghi JSON độc lập. Trong lượt thực nghiệm được báo cáo, công cụ phát sự kiện Vector được bật có kiểm soát rồi dừng để bảo toàn tài nguyên của cụm lab, khiến kết quả MV cuối phản ánh 986 chuyển chứ không phải toàn bộ tệp.

Phạm vi công nghệ được khảo sát gồm Vector (nguồn phát sự kiện), Redpanda (broker tương thích Kafka), RisingWave (streaming SQL engine với materialized view), MinIO kết hợp Apache Iceberg (object storage và table format), Argo CD (công cụ GitOps cho Kubernetes), VictoriaMetrics (cơ sở dữ liệu time series), Grafana (lớp trực quan hóa metric), và K3s (bản phân phối Kubernetes nền). Phạm vi đánh giá cutover được giới hạn ở tên truy vấn public `mv_zone_stats`; báo cáo không kiểm chứng rằng đường ghi xuống Iceberg sink phía sau tự chuyển sang logic green nhờ phép đổi tên materialized view.

1.5. Câu hỏi nghiên cứu

1.6. Đóng góp và cấu trúc báo cáo

Đề tài đóng góp bốn nội dung gắn kết với nhau:

- Một artifact streaming lakehouse có thể tái hiện trên cụm K3s ba server và được triển khai bằng GitOps.
- Một kịch bản Blue/Green MV swap được đo trong khi public query vẫn chạy liên tục.
- Bộ dashboard quan sát kèm cách diễn giải cho cutover, tài nguyên và tính toàn

ven dữ liệu.

- Phân tích giới hạn bằng chứng, tách rõ năng lực công nghệ được tài liệu hóa khỏi kết quả đã đo tại artifact.

Chương 2 trình bày cơ sở lý thuyết về Data Lakehouse, stream processing, materialized view, GitOps, observability và bài báo nền tảng Ursa. Chương 3 mô tả phương pháp Design Science Research, kiến trúc Continuum, topology cụm K3s, đối tượng SQL và thiết kế Blue/Green. Chương 4 trình bày kết quả thực nghiệm, bao gồm phân tích chất lượng dữ liệu nguồn, replay ingest, cutover và quan sát tài nguyên. Chương 5 thảo luận kết quả, giới hạn và kết luận. Phần phụ lục cung cấp SQL cốt lõi, cấu hình trích yếu, chỉ mục bằng chứng và bảng thuật ngữ chi tiết.

Chương 2. Tổng quan và cơ sở lý thuyết

2.1. Data Lakehouse và Apache Iceberg

Data warehouse thường đưa dữ liệu vào các bảng được quản trị chặt để phục vụ truy vấn phân tích ổn định: schema được kiểm soát, các phép biến đổi ETL (*Extract, Transform, Load*) thực hiện trước khi dữ liệu vào kho và engine truy vấn hoạt động trên dữ liệu đã được làm sạch. Ngược lại, *data lake* lưu các tệp đa dạng (CSV, JSON, Parquet, Avro...) trên storage chi phí thấp mà không bắt buộc lược đồ chung; do đó dữ liệu dễ vào kho nhưng quản trị ở lớp bảng có thể thiếu. Mô hình *lakehouse* kết hợp hai hướng: dữ liệu vẫn nằm trên định dạng mở của data lake, nhưng được bổ sung một lớp bảng có *metadata*, lịch sử phiên bản và ngữ nghĩa giao dịch theo nguyên tắc ACID (*Atomicity, Consistency, Isolation, Durability*), để nhiều engine khác nhau có thể đọc cùng một kết quả nhất quán [3].

Object storage là lớp lưu trữ trong đó dữ liệu được truy cập theo khóa (*key*) qua API HTTP, thường tương thích giao diện Amazon S3, thay vì qua đường dẫn tệp truyền thống; mỗi object là một tập hợp byte kèm metadata. Trong artifact này, MinIO cung cấp endpoint tương thích S3 cho cả checkpoint lẫn bảng Iceberg. Apache Iceberg không phải một cơ sở dữ liệu chạy truy vấn mà là một *table format*: đặc tả mô tả những tệp dữ liệu nào thuộc một bảng, *snapshot* nào (ảnh chụp trạng thái bảng tại một thời điểm) là hiện hành, và *manifest* nào cho phép theo dõi thay đổi [1]. Manifest ở đây là tệp metadata trung gian liệt kê các tệp dữ liệu. Iceberg phân biệt hai loại object xóa: *equality-delete* đánh dấu xóa theo giá trị khóa, còn *position-delete* đánh dấu xóa theo vị trí dòng trong tệp. Hai cơ chế này phục vụ cập nhật theo kiểu upsert. Vì vậy, khi báo cáo nói “đầu ra lakehouse tồn tại”, bằng chứng cụ thể là các tệp Parquet và metadata Iceberg do sink ghi trên MinIO, không phải khẳng định rằng MinIO tự xử lý SQL.

2.2. Stream processing, state và materialized view

Một *event* trong pipeline của đề tài là bản ghi chuyến taxi được phát lại dưới dạng thông điệp JSON. Xử lý luồng cập nhật kết quả tại thời điểm event đến, thay vì chờ toàn bộ tệp hoàn thành như xử lý batch. Đối tượng phục vụ kết quả gần thời gian thực ở đây là *materialized view* (MV): trong khi một view thông thường chỉ là định nghĩa truy vấn được chạy lại mỗi lần đọc, một *materialized view* là kết quả truy vấn đã được hệ thống duy trì sẵn và cập nhật theo thay đổi đầu vào; người đọc nhận ngay bảng tổng hợp hiện hành mà không phải tính lại từ dữ liệu thô. Với dữ liệu taxi của đề tài, MV

được nhóm theo borough (quận) và zone (khu vực TLC) và cung cấp các trường `trip_count` (số chuyến), `total_fare` (tổng cước) và `avg_distance` (quãng đường trung bình).

Phép join với bảng tra cứu khu vực và phép aggregation theo vùng đều cần *state*, tức thông tin trung gian phải được giữ qua nhiều event để cập nhật chính xác (ví dụ tổng và đếm hiện thời của mỗi vùng). *Checkpoint* là điểm lưu trạng thái có chủ đích để engine có cơ sở tiếp tục hoặc khôi phục xử lý sau gián đoạn. Continux cấu hình RisingWave dùng bucket `rw-checkpoint` trên MinIO cho state/checkpoint, nhờ đó trạng thái xử lý được đặt tách khỏi vòng đời pod tính toán.

2.3. Broker Kafka-compatible và RisingWave

Kafka đã định hình mô hình log phân tán cho streaming. Trong mô hình này, *topic* là một luồng thông điệp có tên (giống như một “ống” chứa các message cùng loại), *partition* là phần chia của topic phục vụ thứ tự cục bộ và xử lý song song, và *offset* là vị trí thứ tự của một thông điệp trong một partition [7]. *Producer* là phía ghi message vào topic, còn *consumer* là phía đọc; consumer ghi nhớ offset đã đọc để có thể tiếp tục mà không đọc lại. Continux dùng Redpanda, một broker được viết lại bằng C++ nhưng tương thích Kafka API, để tách công cụ phát replay Vector (producer) khỏi RisingWave (consumer) [11]. Nhờ lớp trung gian này, phía phát replay và phía tính toán không cần chạy cùng nhịp; nếu RisingWave xử lý chậm hơn Vector phát, message vẫn được Redpanda đệm trong topic.

Topic được dùng tên `nyc-taxi-events` với ba partition và một replica. Ba partition mô tả cách luồng được chia để có thể xử lý song song. Một replica có nghĩa là hệ thống không đo khả năng chịu mất broker node như một cấu hình production HA; nếu mất broker node, dữ liệu trong topic không có bản sao dự phòng.

RisingWave tiếp nhận source Kafka-compatible, join với bảng lookup đặt trên storage tương thích S3, duy trì các materialized view và tạo Iceberg sink để ghi kết quả ra lakehouse. Trong Continux, sink này ghi bảng tổng hợp ra prefix Iceberg trên MinIO để người vận hành có thể kiểm tra object dữ liệu và metadata sau replay.

2.4. Blue/Green MV swap

Blue/Green deployment là một mẫu triển khai trong đó hai phiên bản của cùng một dịch vụ được duy trì song song: *blue* là phiên bản đang phục vụ thực tế, còn *green* là ứng viên thay thế đã chuẩn bị sẵn nhưng chưa được public truy cập.

Khi ứng viên đạt yêu cầu, “*cutover*” là thao tác chuyển điểm truy cập công khai từ blue sang green; nếu chỉ trong ngữ cảnh tên, “*swap*” là phép hoán đổi định danh giữa hai phiên bản. Trong Continux, đơn vị được chuyển không phải pod Kubernetes mà là định nghĩa MV: `mv_zone_stats` là tên public ổn định mà client tiếp tục gọi sau *cutover*; `mv_zone_stats_blue` (baseline) giữ tất cả record đã join được; `mv_zone_stats_green` loại các chuyến có `fare_amount` hoặc `trip_distance` âm.

RisingWave hỗ trợ cú pháp `ALTER MATERIALIZED VIEW ... SWAP WITH ...` để đổi tên hai MV [13]. Tính “atomic” ở đây chỉ áp dụng cho thao tác đổi tên nhìn từ truy vấn public trong engine, nghĩa là client thấy việc đổi tên là tức thời và không có trạng thái trung gian; nó không tự động chứng minh rằng mọi consumer hoặc sink đã đổi dependency theo. Tài liệu về thay đổi streaming job lưu ý các downstream dependency như sink cần được xem xét riêng khi logic upstream thay đổi [12]. Do đó kết luận của nghiên cứu chỉ áp dụng tại endpoint truy vấn public, không mở rộng sang hành vi của Iceberg sink phía sau.

2.5. GitOps và observability

OpenGitOps xác định bốn nguyên tắc cho trạng thái mong muốn của hệ thống: phải có tính *khai báo* (mô tả “muốn gì” thay vì “làm như thế nào”), được *phiên bản hóa* trong Git, được *agent* tự động kéo về, và được *reconcile* liên tục. Reconcile nghĩa là agent so sánh trạng thái chạy thật với trạng thái mong muốn rồi đưa hệ thống về đúng cấu hình [10]. Continux dùng Argo CD theo mẫu *App-of-Apps*, trong đó một Application gốc trên Argo CD trở tới một thư mục chứa các Application con; cách tổ chức này giúp toàn bộ cụm được khởi tạo và đồng bộ chỉ bằng một điểm vào duy nhất [2]. Việc scale Vector trong một lượt replay là thao tác đo có chủ đích diễn ra ngoài đường GitOps, không thay thế vai trò của Git làm nguồn cấu hình dài hạn.

Observability là khả năng suy luận trạng thái nội tại của hệ thống từ tín hiệu đo bên ngoài như metric, log và trạng thái workload. *Metric* là một chuỗi số có dấu thời gian, gọi là *time series*, mô tả một đại lượng theo thời gian; metric thường được *scrape* (cào lấy) bởi một backend từ endpoint HTTP do workload hoặc một *exporter* (thành phần dịch chỉ số nội bộ sang định dạng metric chuẩn) phát ra. VictoriaMetrics đảm nhiệm vai trò backend time-series cho artifact, lưu metric theo thời gian [17]; Grafana đảm nhiệm trực quan hóa qua các dashboard [5]; còn exporter của Continux chuyển các SQL count và kết quả *cutover* thành metric chuẩn để VictoriaMetrics

Bảng 2.1: Vai trò các lớp công nghệ trong Continux

Lớp	Công nghệ	Vai trò được đánh giá
Orchestration	K3s, Argo CD	Chạy workload và đồng bộ cấu hình
Ingestion	Vector	Replay JSONL có kiểm soát
Event broker	Redpanda	Topic Kafka-compatible
Stream compute	RisingWave	Join, aggregation, public/blue/green MV
Lakehouse storage	MinIO, Iceberg	Checkpoint và object bảng đầu ra
Observation	VictoriaMetrics, Grafana, exporter	Count, swap, health và resource

scrape. Dashboard giúp đọc hiện tượng, nhưng kết luận trong báo cáo luôn được đối chiếu với kết quả SQL thực và listing object trên MinIO để tránh diễn giải quá mức một panel lỗi hoặc panel có dữ liệu thiếu.

2.6. Ursa và nghiên cứu liên quan

Ursa là streaming engine leaderless, Kafka-compatible và lakehouse-native, tập trung giảm chi phí cross-AZ, disk replication và connector overhead, đồng thời báo cáo throughput và semantics của thiết kế engine [8]. Continux không triển khai Ursa hay benchmark chi phí; đóng góp bổ sung là khảo sát GitOps, observability và thay đổi logic public query trên stack triển khai nhỏ.

Flink hỗ trợ xử lý batch và stream trong một engine [4]; Kafka Streams và ksqlDB đại diện hướng xử lý gắn hệ sinh thái Kafka. RisingWave được chọn do giao diện SQL/MV và cú pháp swap phù hợp artifact. Đây là định vị khái niệm, không phải benchmark đối đầu.

Chương 3. Nghiên cứu thực nghiệm và kiến trúc hệ thống

3.1. Phương pháp nghiên cứu

Nghiên cứu áp dụng phương pháp Design Science Research (DSR) [6], một khung phương pháp luận cho nghiên cứu hệ thống thông tin trong đó tri thức được tạo ra thông qua việc xây dựng và đánh giá những hiện vật kỹ thuật. Theo DSR, một *artifact* là hiện vật kỹ thuật được thiết kế có chủ đích để xử lý một vấn đề cụ thể; ở đây artifact là toàn bộ kiến trúc Continux gồm hạ tầng K3s, pipeline streaming từ Vector đến Iceberg, lớp GitOps Argo CD và lớp quan sát VictoriaMetrics/Grafana. *Evaluation* là pha đánh giá artifact bằng lượt thực nghiệm có kiểm soát, thay vì chỉ mô tả cấu hình kỳ vọng trên giấy.

Bảng chứng dùng cho quá trình đánh giá gồm kết quả lệnh dạng văn bản, metric ở định dạng JSON do VictoriaMetrics trả về, và ảnh dashboard Grafana. Bộ kết quả sinh trên host triển khai (node imac) đã được sao chép về máy phân tích Windows tại thư mục `evidence/finalize/20260522-151720/` để đối chiếu trong khi viết báo cáo. Đánh giá chỉ sử dụng đúng trạng thái, số đếm và thời gian có trong bảng chứng; các mục không có phép đo tương ứng, như benchmark theo ba mức tải đã định nghĩa trong repo, được xác định là giới hạn của nghiên cứu chứ không được nội suy.

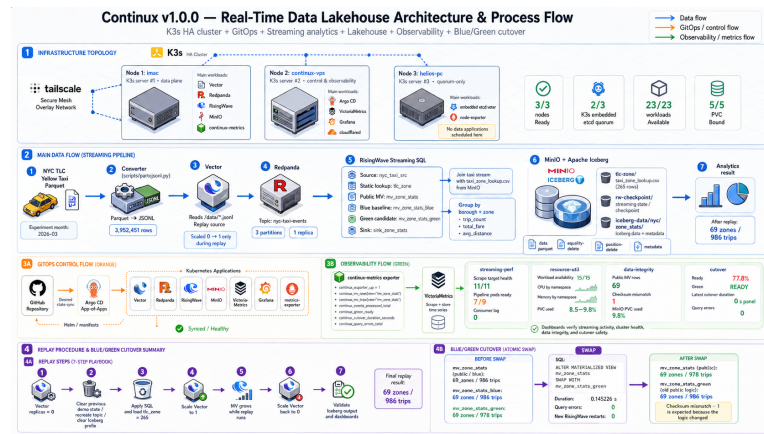
3.2. Dữ liệu và chuẩn bị luồng

Sở Taxi và Dịch vụ vận tải New York, gọi tắt là NYC TLC, công bố trip record công khai cho taxi và xe vận tải tại New York; đây là một trong những nguồn dữ liệu vận tải đô thị được sử dụng nhiều nhất trong nghiên cứu phân tích [9]. Continux sử dụng dữ liệu Yellow Taxi tháng 2026-03. Một script được viết để chuyển tệp Parquet nguồn, định dạng cột nén tối ưu cho phân tích lô, sang JSONL nhằm tạo dòng sự kiện có thứ tự cho luồng replay. Bảng tra cứu khu vực TLC Taxi Zone gồm 265 dòng được giữ ở dạng CSV trên MinIO; mỗi dòng ánh xạ một mã `location_id` sang `borough` (quận) và `zone`.

Mỗi dòng JSONL là biểu diễn tuần tự hóa của một chuyến taxi; khi Vector phát dòng đó vào broker, nó trở thành event trong lượt replay. Vector bổ sung hai trường meta khi phát: `event_id` (định danh duy nhất cho mỗi event được phát) và `event_time` (mốc thời gian phát thực tế của Vector). Trong manifest đang triển khai, tốc độ phát được giới hạn ở 2 sự kiện mỗi giây để cụm lab tài nguyên hạn chế xử lý ổn định; Vector được dừng sau khi đủ quan sát. Do đó kết quả MV không phản ánh

Bảng 3.1: Nguồn dữ liệu thực nghiệm

Thuộc tính	Giá trị
Loại chuyển và tháng	Yellow Taxi, 2026-03
Định dạng nguồn/đích	Parquet / JSONL
Số dòng JSONL xác nhận	3.952.451
Lookup	TLC Taxi Zone, 265 dòng
Lượt đo báo cáo	Replay có kiểm soát, public MV cuối 986 trips



Hình 3.1: Kiến trúc và luồng quy trình Continuum.

việc xử lý toàn bộ JSONL, dù số dòng của tệp nguồn đã được xác nhận độc lập bằng lệnh `wc -l` trên host `imac`; phân tích chi tiết hơn về chất lượng bộ dữ liệu thô được trình bày ở Mục 4.3.

3.3. Kiến trúc tổng thể

Kiến trúc trong Hình 3.1 có ba luồng. Trong luồng dữ liệu, file Parquet được chuyển sang JSONL để Vector replay từng event, Redpanda đệm luồng và RisingWave duy trì kết quả SQL trước khi sink ghi các object Iceberg trên MinIO. Trong luồng điều khiển, GitHub giữ manifest còn Argo CD triển khai và kiểm tra sai lệch cấu hình trên Kubernetes. Trong luồng quan sát, workload và exporter phát metric để VictoriaMetrics lưu theo thời gian và Grafana hiển thị.

Việc tách ba luồng này làm rõ phạm vi phép đo: đầu ra Iceberg cho biết đường ghi dữ liệu đã vận hành; trạng thái Argo CD cho biết cấu hình chính được reconcile; còn query loop và metric cutover cho biết tên truy vấn public có tiếp tục phục vụ trong thao tác đổi logic hay không.

Bảng 3.2: Topology cụm K3s thực nghiệm

Node	Tài nguyên ghi nhận	Vai trò
imac	Intel i5-8500, 6 cores, 8 GB RAM, 200 GB SSD, Ubuntu 26.04	Server #1; Vector, Redpanda, RisingWave, MinIO, exporter
continux-vps	2 vCPU, 4 GB RAM, 80 GB SSD, Ubuntu 24.04	Server #2; Argo CD, VictoriaMetrics, Grafana
helios-pc	WSL Ubuntu 26.04, host Intel i5-12500H, 16 GB RAM	Server #3; quorum-only, node-exporter

3.4. Topology triển khai

Các node giao tiếp qua *Tailscale*, một mesh VPN xây trên giao thức WireGuard, tạo lớp mạng riêng overlay giữa các máy ở các mạng khác nhau như thể chúng nằm trong cùng một mạng LAN. K3s ở chế độ HA dùng *embedded etcd*: etcd là cơ sở dữ liệu key-value phân tán giữ trạng thái cụm Kubernetes, và “embedded” nghĩa là etcd chạy bên trong tiến trình K3s thay vì tách rời. Cụm yêu cầu *quorum 2/3*, tức cần ít nhất hai trên ba node etcd còn hoạt động để các thao tác ghi đạt đồng thuận; vì vậy node *helios-pc* chỉ giữ vai trò quorum (bỏ phiếu) mà không nhận workload dữ liệu. Workload tính toán và lưu trữ được đặt trên *imac* để tận dụng SSD và RAM cục bộ; *continux-vps* đảm nhiệm control plane và observability.

Vì vậy, thuật ngữ HA trong báo cáo chỉ nói về lớp Kubernetes control plane và etcd. Artifact không được cấu hình như một pipeline HA từ đầu đến cuối: Redpanda chạy một broker (`statefulset.replicas=1`), topic `nyc-taxi-events` có `replicationFactor=1`, và các thành phần dữ liệu dùng cấu hình lab tiết kiệm tài nguyên.

3.5. Đối tượng SQL và Blue/Green design

Bảng 3.3: Đối tượng dữ liệu cốt lõi

Object	Input	Chức năng
tlc_zone	CSV trên MinIO	Lookup vị trí sang borough/zone
nyc_taxi_src	Redpanda topic	Source event JSON
mv_zone_stats_blue	Source + lookup	Aggregation baseline
mv_zone_stats	Blue MV ban đầu	Tên public được query
mv_zone_stats_green	Source + lookup + filter	Candidate logic
sink_zone_stats	Public MV lúc tạo sink	Đầu ra Iceberg upsert

Tên public `mv_zone_stats` đóng vai trò hợp đồng truy vấn cho client: client không cần đổi SQL khi logic phục vụ chuyển từ blue sang green. Ngược lại, `sink_zone_stats` được tạo dựa trên dependency đã tồn tại trước swap; vì vậy việc public MV trả kết quả green sau cutover không đủ để suy ra sink Iceberg đã tự chuyển cùng logic.

Listing 1: Điều kiện bổ sung trong logic green

```
WHERE t.fare_amount >= 0
AND t.trip_distance >= 0
```

Sau khi green MV sẵn sàng, query loop gọi public MV với chu kỳ 0,5 s, đồng thời hệ thống thực hiện:

Listing 2: Câu lệnh Blue/Green MV cutover

```
ALTER MATERIALIZED VIEW mv_zone_stats
SWAP WITH mv_zone_stats_green;
```

3.6. Thang đo đánh giá

Bảng 3.4: Nhóm thang đo và cách diễn giải

Nhóm	Bằng chứng	Diễn giải
Cutover	Duration, query errors, restart, counts trước/sau	Swap public MV thành công, không ghi nhận lỗi query
Tính toàn vẹn quan sát được	MV counts, filter difference, objects	Output tồn tại; chênh lệch phải phù hợp logic
Streaming activity	Replay progress, dashboard, lag hiển thị	Chứng minh hoạt động, không phải benchmark tổng quát
Resource stability	Nodes, workloads, PVC, CPU/RAM	Cluster đủ vận hành lượt đo

3.7. Nguồn bằng chứng và khả năng truy vết

Bộ bằng chứng cục bộ của lượt đo nằm tại `evidence/finalize/20260522-151720/` và gồm 48 file kết quả văn bản/JSON. Các file baseline ghi node, PVC, Argo CD và mức sẵn sàng RisingWave; nhóm 05-* ghi replay và Iceberg listing; nhóm 06-* ghi green catch-up, query loop, swap, duration và metric được VictoriaMetrics lưu.

Cách đọc bằng chứng tách thời điểm sự kiện khỏi thời điểm thu lại metric. Timestamp swap được ghi trong `06-cutover-duration.txt` là epoch 1779466691, tương ứng 2026-05-22 16:18:11 UTC và nằm trong cửa sổ query loop. File VictoriaMetrics xác nhận lại duration và lỗi truy vấn tại timestamp scrape 1779467656, muộn hơn thao tác swap; timestamp scrape được dùng để chứng minh metric đã được lưu, không thay thế thời điểm cutover. Metadata của bundle nguồn cho thấy file swap/duration được cập nhật muộn hơn log query loop và kết quả sau swap; vì vậy duration được báo cáo như số đo do runbook ghi, không phải latency được tái dựng độc lập từ một trace bất biến.

Chương 4. Kết quả thực nghiệm, đánh giá và bàn luận

4.1. Môi trường và phiên bản

Bảng 4.1: Phiên bản phần mềm được ghi trong repo

Thành phần	Phiên bản
K3s	v1.35.5+k3s1
Helm	v4.1.4
Argo CD	v3.4.2
Tailscale	v1.98.2
Redpanda	v26.1.8
RisingWave	v2.8.3
Vector	0.55.0-alpine
VictoriaMetrics	v1.143.0
Grafana	v13.0.1+security-01
cloudflared	2026.5.0

4.2. Cluster, GitOps và mức sẵn sàng pipeline

Bảng 4.2: Kết quả hạ tầng và pipeline

Chỉ số	Kết quả
K3s nodes Ready	3/3
PVC Bound	5/5
Workloads Available sau replay	23/23
Argo CD applications chính	Synced/Healthy
RisingWave roles	meta, compute, compactor, frontend RUNNING
TLC lookup table	265 dòng
Iceberg objects	data Parquet, equality-delete, position-delete, metadata

4.3. Phân tích chất lượng dữ liệu nguồn

Trước khi diễn giải kết quả replay, đề tài đối chiếu chính xác bộ dữ liệu thô để giải thích vì sao logic green được thiết kế như đã trình bày. Toàn bộ tệp `data/raw/yellow_tripdata_2026-03.jsonl` được quét tuần tự một lần trên host `imac` bằng script Python, không lấy mẫu, để đếm các trường hợp vi phạm chất lượng và tính phân phối cơ bản của `fare_amount`, `trip_distance`, `pu_location_id` và `pickup_time`. Kết quả được tổng hợp ở Bảng 4.3.

Bảng 4.3: Phân tích chất lượng bộ dữ liệu NYC TLC Yellow Taxi tháng 2026-03 sau chuyển đổi JSONL

Chỉ số quan sát được trên bộ dữ liệu thô	Giá trị
Tổng số bản ghi JSONL	3.952.451
Bản ghi có <code>fare_amount < 0</code>	20.724 (0,5243%)
Bản ghi có <code>trip_distance < 0</code>	0
Bản ghi có <code>fare_amount = 0</code>	2.995
<code>fare_amount</code> min / max	-990,00 / 1.843,30 USD
<code>fare_amount</code> mean / median	21,33 / 15,60 USD
<code>trip_distance</code> min / max	0 / 288.381,68 miles
<code>trip_distance</code> mean / median	5,98 / 1,82 miles
<code>pu_location_id</code> xuất hiện trong bộ dữ liệu	261/265 mã
Khoảng <code>pickup_time</code>	2008-12-31 đến 2026-04-01 (35 ngày phân biệt)
Giờ cao điểm theo <code>pickup_time</code>	18:00 với 279.487 chuyến
Giờ thấp điểm theo <code>pickup_time</code>	04:00 với 35.646 chuyến

Kết quả này có ba ý nghĩa cho thiết kế và đánh giá:

- Bộ lọc `fare_amount >= 0` của logic green được đặt trên một hiện tượng có thật: 20.724 bản ghi với cước âm, chiếm 0,5243% bộ dữ liệu thô. Tỷ lệ nhỏ này phù hợp với chênh lệch 8 chuyến giữa blue (986) và green (978) đo được trên mẫu nhỏ của lượt replay; ở mức 986 chuyến, kỳ vọng số bản ghi cước âm là

khoảng $986 \times 0,5243\% \approx 5$ chuyến.

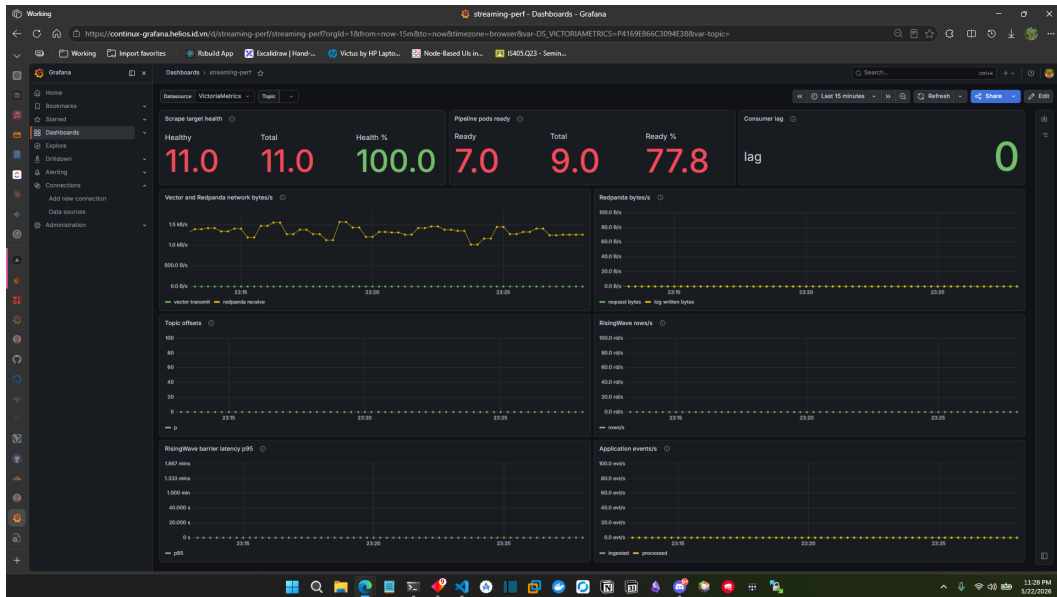
- Không có bản ghi nào với `trip_distance < 0` trong bộ dữ liệu; vì vậy predicate `trip_distance >= 0` trong logic green là biện pháp phòng ngừa cho các tháng dữ liệu khác hoặc các bộ dữ liệu tương tự.
- `trip_distance` có giá trị tối đa 288.381,68 miles và `pickup_time` chứa các mốc ngoài tháng 2026-03, từ 2008-12-31 đến 2026-04-01. Đây là các bất thường ở dữ liệu/schema thô chưa được logic green hiện tại xử lý và được liệt kê tại Mục 4.9.

Phân tích này cũng giải thích tại sao MV chỉ chứa 69 cặp (borough, zone) chứ không phải toàn bộ 261 mã `pu_location_id` mà bộ dữ liệu chạm vào: trong cửa sổ replay đã đo, Vector chỉ đưa được một mẫu 986 chuyến qua broker trước khi được dừng có chủ đích, và mẫu nhỏ này không phủ hết tất cả các vùng. Bối cảnh giờ cao điểm 18:00 và thấp điểm 04:00 minh họa tính chất tải biến thiên theo thời gian của ITS và củng cố luận điểm cần observability gần thời gian thực; tuy nhiên các con số này thuộc cấp bộ dữ liệu, không phải kết quả đo throughput của artifact.

4.4. Replay ingest và đầu ra lakehouse

Vector được mở trong khoảng có kiểm soát rồi đưa về `replicas=0`. File `05-mv-before-stop.txt` ghi public MV ở 66 zones / 912 trips; sau khi các event đã nhận được xử lý hết, `05-mv-final-count.txt` ghi trạng thái ổn định 69 zones / 986 trips. File `05-minio-iceberg-after-replay.txt` liệt kê đầu ra Iceberg mới, cho thấy đường ghi lakehouse đã hoạt động trong lượt replay.

Kết quả này không đại diện throughput tối đa hay xử lý toàn bộ 3.952.451 dòng. Các file cấu hình mức tải trong repo không kèm kết quả thực nghiệm, nên không được trình bày thành benchmark.



Hình 4.1: Dashboard hiệu năng streaming sau lượt replay/cutover. Consumer lag hiển thị 0 trong cửa sổ ảnh.

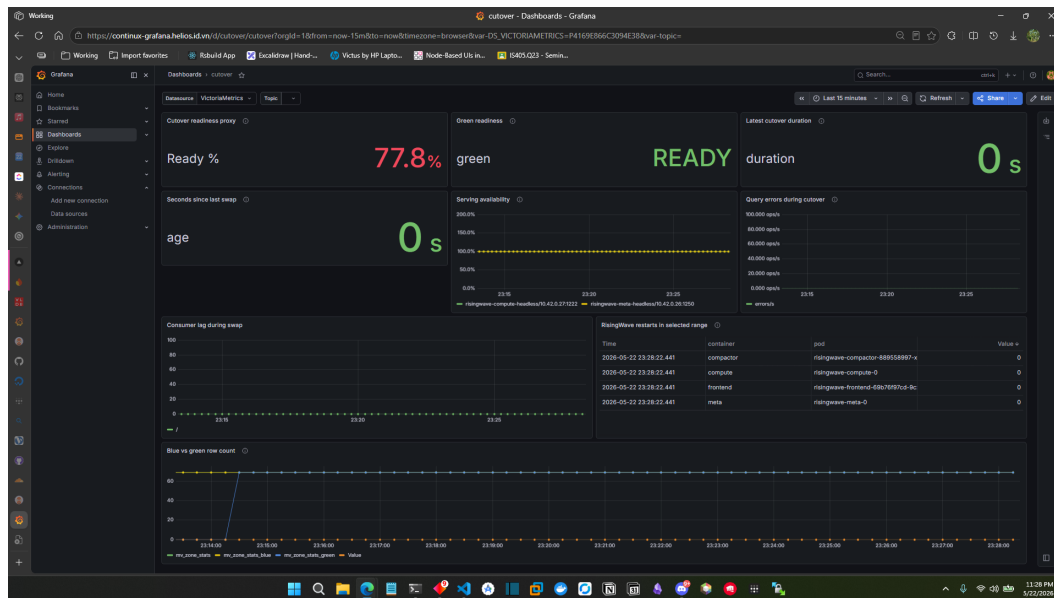
4.5. Blue/Green MV cutover

Bảng 4.4: Kết quả Blue/Green MV swap

Chỉ số	Giá trị
Public MV trước swap	69 zones / 986 trips
Blue MV trước swap	69 zones / 986 trips
Green MV trước swap	69 zones / 978 trips
Cutover command	ALTER MATERIALIZED VIEW ... SWAP WITH ...
Thời gian do runbook ghi	0,145226 s
Lỗi query trong loop	0
RisingWave restarts trong swap	0
Public MV sau swap	69 zones / 978 trips
View mang tên green sau swap	69 zones / 986 trips

Chênh lệch 8 trips giữa blue và green phù hợp điều kiện green loại record có `fare_amount < 0` hoặc `trip_distance < 0`. Bảng chứng không cho thấy chênh lệch này phát sinh từ lỗi chuyển giao.

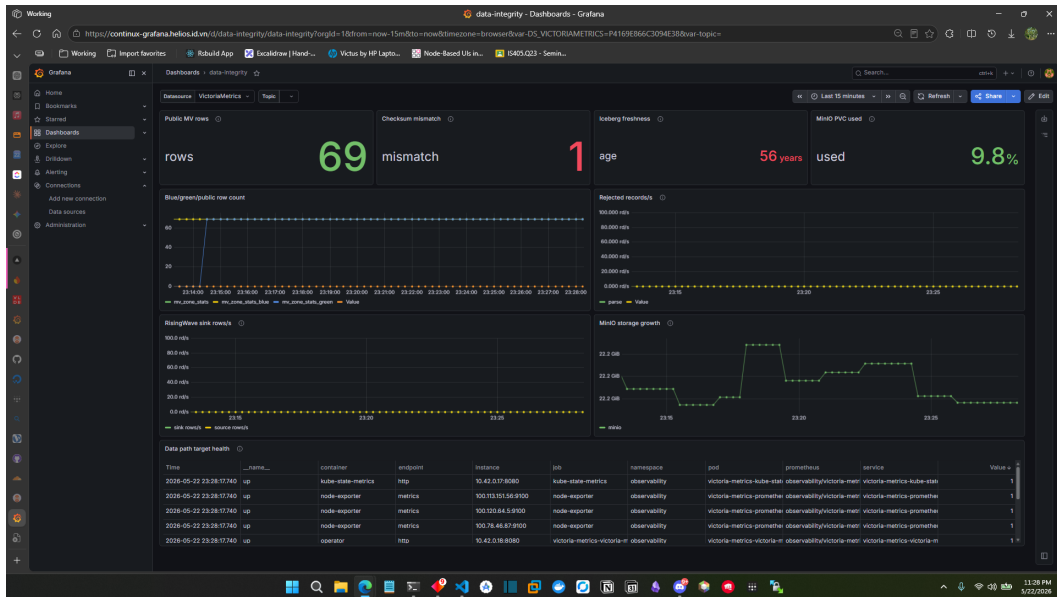
Query loop trong `06-query-loop-during-cutover.txt` ghi các truy vấn thành công khi public MV chuyển từ trạng thái `69|986` sang `69|978`, trong khoảng từ `16:17:36 UTC` đến `16:18:16 UTC` ngày `2026-05-22`; log không có dòng `ERROR`. File `duration` ghi timestamp swap `16:18:11 UTC` và duration `0,145226 s`; hai file `VictoriaMetrics` thu sau đó xác nhận metric `duration` và `continuum_query_errors_total = 0` đã được lưu. Do file `swap/duration` trong bundle nguồn có mốc cập nhật sau log chuyển trạng thái, số `duration` được dùng như phép đo runbook đã ghi nhận gắn với transition quan sát, chưa phải latency được kiểm chứng lại độc lập. Bằng chứng hỗ trợ nhận định không ghi nhận gián đoạn truy vấn, chứ không chứng minh mọi client hoặc mọi lỗi mạng đều vắng mặt.



Hình 4.2: Dashboard cutover: green ready, query errors bằng 0 và restart bằng 0; panel `duration` làm tròn giá trị nhỏ hơn một giây.

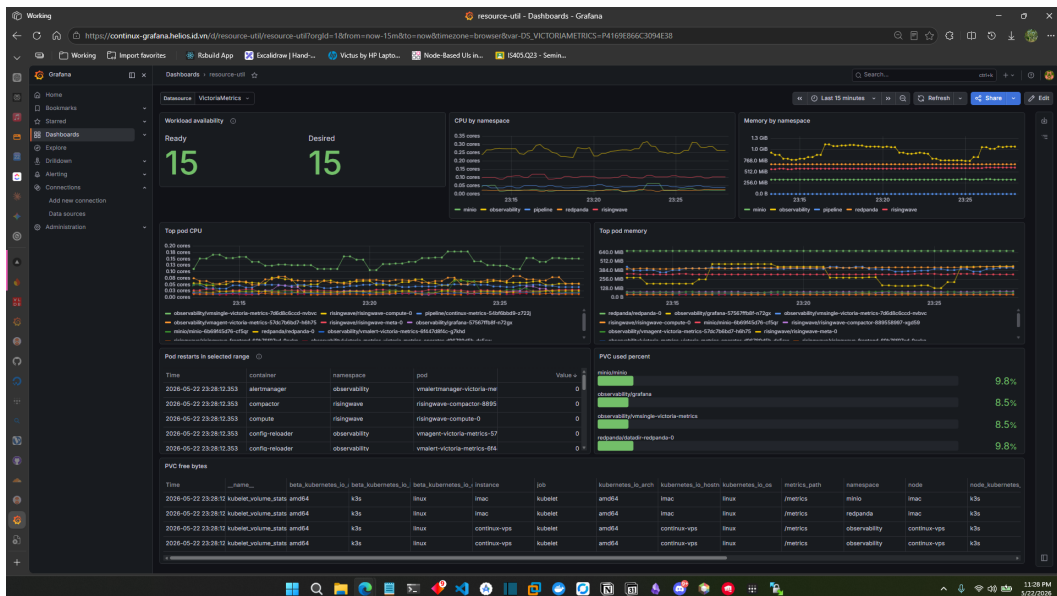
4.6. Tính toàn vẹn quan sát được và tài nguyên

Dashboards tính toàn vẹn dữ liệu hiển thị public MV có 69 dòng và checksum mismatch 1 sau swap. Exporter so public với logic baseline; sau cutover public mang green logic, nên mismatch là kết quả mong đợi của thay đổi định nghĩa. Panel “Iceberg freshness” hiển thị “56 years”; do metric timestamp chưa hợp lệ trong cấu hình quan sát, giá trị này không được dùng làm kết quả.



Hình 4.3: Dashboard tính toán vẹn dữ liệu sau swap; mismatch được diễn giải theo thay đổi logic.

Dashboard tài nguyên ghi workload availability 15/15 ở thời điểm ảnh và PVC MinIO/Redpanda khoảng 9, 8%. Kết quả cục bộ trong 05-k3s-overview-after-replay.txt ghi tình trạng toàn cluster sau replay là 23/23 workload available, RAM imac khoảng 49% và disk root khoảng 11%.



Hình 4.4: Dashboard resource utilization trong cửa sổ quan sát thực nghiệm.

4.7. Tổng hợp đánh giá

Bảng 4.5: Tổng hợp kết luận theo bằng chứng

Nhóm	Bằng chứng ghi nhận	Kết luận
Query-layer cutover	Runbook duration 0,145226 s, lỗi 0, restart 0	Zero-downtime quan sát được cho public MV
Đầu ra lakehouse	Iceberg objects sau replay	Sink đã ghi đầu ra trong replay
Logic difference	Green ít hơn blue 8 trips	Thay đổi bộ lọc được quan sát
Cluster stability	3/3, 5/5, 23/23	Đủ chạy kịch bản đã đo
Multi-load performance	Không có run results	Không kết luận

4.8. Thảo luận kết quả

Kết quả trả lời tích cực RQ1 trong phạm vi quan sát: tên MV public được chuyển từ baseline sang candidate mà query loop không ghi nhận lỗi. Duration 0,145226 s do runbook ghi là chỉ báo vận hành cho thay đổi SQL nhỏ trên cụm lab; do nguồn gốc và thời điểm file chưa cho phép tái dựng latency độc lập, con số này không phải SLA hay bằng chứng cho mọi mẫu truy vấn.

RQ2 được trả lời qua đầu ra Parquet/metadata trên MinIO sau replay. Tuy nhiên, điều này chỉ chứng minh sink hoạt động trong pipeline đã chạy; bởi tài liệu RisingWave mô tả swap là đổi tên MV và yêu cầu xem xét riêng các dependency, nghiên cứu không khẳng định sink đã đổi sang green sau swap.

RQ3 được trả lời bằng sức khỏe cụm và dashboard tài nguyên. Rate limit thấp và việc dùng Vector là lựa chọn bảo vệ data plane 8 GB RAM; kết quả phản ánh tính khả thi chức năng, không phải công suất tối đa.

So với Ursa, Continux không đề xuất engine mới. Giá trị bổ sung là nổi động lực streaming lakehouse với GitOps, bằng chứng vận hành và đổi logic public query trên môi trường nhỏ có thể trình diễn.

4.9. Hạn chế

Bảng 4.6: Giới hạn bằng chứng và tác động tới kết luận

Giới hạn	Tác động
Lượt replay chỉ dừng ở 986 trips	Không suy rộng thành xử lý toàn bộ dữ liệu hoặc throughput tối đa
Không có run ba mức tải	Không báo cáo latency/throughput theo tải
Sink là dependency tạo trước MV swap	Không kết luận Iceberg table chuyển sang green logic
Kafka catalog/Iceberg freshness metric chưa đầy đủ	Dựa trên SQL count và MinIO listing cho kết luận hiện hữu
File duration/swap được cập nhật sau log chuyển trạng thái	Báo duration như số đo runbook, không coi là latency tái dựng độc lập
Redpanda chạy một broker và topic có replicationFactor=1	Không đánh giá khả năng HA hoặc chịu lỗi của broker và data path khi node lỗi
Dataset thô chứa timestamp ngoài tháng 2026-03 và outlier cực đoan của trip_distance	Logic green hiện tại chỉ lọc giá trị âm; bộ lọc theo phạm vi thời gian và phạm vi quãng đường chưa được áp dụng

Chương 5. Kết luận

Continuum xây dựng thành công một artifact Data Lakehouse thời gian thực trên K3s gồm ingest, broker, streaming SQL, Iceberg object storage, GitOps và observability. Từ bằng chứng hiện có, hệ thống thực hiện một lượt replay tạo public MV và đầu ra Iceberg; trong kịch bản Blue/Green MV swap, query loop ghi 0 lỗi và không có restart RisingWave, còn runbook ghi duration 0,145226 s.

Đóng góp thực nghiệm nổi bật là minh chứng cho cập nhật logic ở **lớp truy vấn public** mà không ghi nhận gián đoạn trong cửa sổ thử nghiệm. Báo cáo cũng ghi nhận pipeline tạo được đầu ra Iceberg và duy trì trạng thái workload ổn định trong lượt đo. Các kết luận được giới hạn trong đúng bằng chứng đã thu thập.

Chương 6. Hướng phát triển

Các nghiên cứu tiếp theo có thể đi theo ba hướng bổ sung lẫn nhau:

- Benchmark nhiều mức tải trên một data plane lớn hơn 8 GB RAM, dùng đúng các kịch bản đã định nghĩa trong repo (low, medium, high) để thu biểu đồ latency/throughput theo tải.
- Thiết kế cutover bao gồm cả sink downstream, chẳng hạn tạo sink mới gắn vào green MV trước cutover rồi đổi tên sink, hoặc thực hiện compaction/rewrite snapshot Iceberg theo logic mới.
- Mở rộng nguồn dữ liệu sang sự kiện giao thông trực tiếp, ví dụ cảm biến giao thông hoặc dữ liệu GPS xe buýt, thay vì replay bộ dữ liệu có sẵn.

Đây là định hướng cho các phiên bản kế tiếp, không phải phần kết quả của lượt thực nghiệm hiện tại.

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Việt

Không có tài liệu tiếng Việt được trích dẫn trong báo cáo.

Tài liệu tiếng Anh

- [1] Apache Software Foundation. “Apache iceberg documentation,” Accessed: May 24, 2026. [Online]. Available: <https://iceberg.apache.org/docs/latest/>
- [2] Argo Project. “Argo cd documentation,” Accessed: May 24, 2026. [Online]. Available: <https://argo-cd.readthedocs.io/>
- [3] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, “Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics,” in *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*, 2021. [Online]. Available: <https://www.vldb.org/cidrdb/2021/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics.html>
- [4] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, “Apache flink: Stream and batch processing in a single engine,” *IEEE Data Engineering Bulletin*, vol. 38, no. 4, pp. 28–38, 2015. [Online]. Available: <https://sites.computer.org/debull/A15dec/p28.pdf>
- [5] Grafana Labs. “Grafana documentation,” Accessed: May 24, 2026. [Online]. Available: <https://grafana.com/docs/grafana/latest/>
- [6] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. DOI: 10.2307/25148625
- [7] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB Workshop*, 2011. [Online]. Available: <https://cwiki.apache.org/confluence/download/attachments/27822226/Kafka-netdb-06-2011.pdf>
- [8] M. Merli, S. Guo, P. Li, H. Chen, and N. Lu, “Ursa: A lakehouse-native data streaming engine for kafka,” *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 5184–5196, 2025. DOI: 10.14778/3750601.3750636 [Online]. Available: <https://www.vldb.org/pvldb/vol18/p5184-guo.pdf>
- [9] New York City Taxi and Limousine Commission. “Tlc trip record data,” Accessed: May 24, 2026. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [10] OpenGitOps Working Group. “Opengitops principles,” Accessed: May 24, 2026. [Online]. Available: <https://opengitops.dev/>

- [11] Redpanda Data. “Redpanda documentation,” Accessed: May 24, 2026. [Online]. Available: <https://docs.redpanda.com/>
- [12] RisingWave Labs. “Alter a streaming job,” Accessed: May 24, 2026. [Online]. Available: <https://docs.risingwave.com/operate/alter-streaming>
- [13] RisingWave Labs. “Alter materialized view: Swap with,” Accessed: May 24, 2026. [Online]. Available: <https://docs.risingwave.com/sql/commands/sql-alter-materialized-view>
- [14] United Nations. “Goal 11: Sustainable cities and communities,” Accessed: May 24, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal11>
- [15] United Nations. “Goal 8: Decent work and economic growth,” Accessed: May 24, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal8>
- [16] United Nations. “Goal 9: Industry, innovation and infrastructure,” Accessed: May 24, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal9>
- [17] VictoriaMetrics. “Victoriametrics documentation,” Accessed: May 24, 2026. [Online]. Available: <https://docs.victoriametrics.com/>

Phụ lục A. SQL cốt lõi

A.1. Source, lookup và baseline MV

```
CREATE SOURCE IF NOT EXISTS nyc_taxi_src (  
    event_id VARCHAR,  
    event_time TIMESTAMPTZ,  
    pickup_time TIMESTAMPTZ,  
    pu_location_id INT,  
    do_location_id INT,  
    fare_amount DOUBLE PRECISION,  
    trip_distance DOUBLE PRECISION  
) WITH (  
    connector = 'kafka',  
    topic = 'nyc-taxi-events',  
    properties.bootstrap.server = 'redpanda.redpanda:9093',  
    scan.startup.mode = 'earliest'  
) FORMAT PLAIN ENCODE JSON;  
  
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_zone_stats_blue AS  
SELECT z.borough, z.zone, COUNT(*) AS trip_count,  
        SUM(t.fare_amount) AS total_fare,  
        AVG(t.trip_distance) AS avg_distance  
FROM nyc_taxi_src t  
JOIN tlc_zone z ON t.pu_location_id = z.location_id  
GROUP BY z.borough, z.zone;  
  
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_zone_stats AS  
SELECT * FROM mv_zone_stats_blue;
```

A.2. Green MV và swap

```
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_zone_stats_green AS  
SELECT z.borough, z.zone, COUNT(*) AS trip_count,  
        SUM(t.fare_amount) AS total_fare,  
        AVG(t.trip_distance) AS avg_distance  
FROM nyc_taxi_src t  
JOIN tlc_zone z ON t.pu_location_id = z.location_id  
WHERE t.fare_amount >= 0  
        AND t.trip_distance >= 0
```

```
GROUP BY z.borough, z.zone;
```

```
ALTER MATERIALIZED VIEW mv_zone_stats  
SWAP WITH mv_zone_stats_green;
```

A.3. Iceberg sink

```
CREATE SINK IF NOT EXISTS sink_zone_stats FROM mv_zone_stats  
WITH (  
    connector = 'iceberg',  
    type = 'upsert',  
    primary_key = 'borough,zone',  
    catalog.type = 'storage',  
    warehouse.path = 's3://iceberg-data/',  
    database.name = 'nyc',  
    create_table_if_not_exists = 'true',  
    table.name = 'zone_stats'  
);
```

Phụ lục B. Cấu hình vận hành trích yếu

```
# Redpanda topic  
topics:  
  - name: nyc-taxi-events  
    partitions: 3  
    replicationFactor: 1  
    config:  
      retention.ms: "86400000"  
  
# Vector rate in deployed ConfigMap  
rate_limit_num = 2  
rate_limit_duration_secs = 1  
  
# RisingWave state store  
stateStore:  
  s3:  
    enabled: true  
    endpoint: http://minio.minio.svc.cluster.local:9000  
    bucket: rw-checkpoint
```